

Fortnite Performance Dashboard

Database Design

Group 15

Business Problem → Requirement Analysis → System Design → Database Design → Sprint Planning
→ Development

1. What is Database Design?

The process of converting the entities, relationships, and business rules identified in Requirement Analysis into a concrete relational schema — deciding tables, columns, data types, keys, constraints, and relationships so the database is accurate, non-redundant, and efficient to query.

2. Design Goals

- Keep the schema minimal four tables, no unnecessary entities, matching the approved requirements.
- Normalize to at least 3NF so player stats and recommendations are never duplicated or inconsistent.
- Enforce data integrity at the database level (keys, constraints) rather than relying only on application code.
- Support the two 1:1 relationships (User↔Player, Player↔Stats) and the 1:many relationship (Player↔Recommendations) called out in the requirements.
- Design for EF Core Code-First so the schema can be generated and versioned through migrations.

3. Entity-Relationship Overview Four entities cover the current scope. Each Player is linked to exactly one User account and has exactly one live Stats record (overwritten on every sync). A Player can accumulate many Recommendations over time, preserving coaching history.

Relationship	Cardinality	Why
User → Player	1:1	Each account links to exactly one Fortnite profile in this scope.
Player → Stats	1:1	Dashboard always shows the latest snapshot; sync overwrites, not appends.
Player → Recommendations	1:Many	Coaching history must be kept so trends and past advice remain visible.

4. Table Design

4.1 Users Stores login credentials and role for both Players and Administrators.

Column	Type	Constraint	Purpose
UserId	INT (Identity)	PK	Unique user identifier.
Name	NVARCHAR(100)	NOT NULL	Display name.
Email	NVARCHAR(256)	UNIQUE, NOT NULL	Login identifier.
Password-Hash	NVARCHAR(512)	NOT NULL	Hashed credential — never stored as plain text.
Role	NVARCHAR(20)	CHECK IN ('Player','Administrator')	Drives role-based access control.
Created Date	DATETIME2	DEFAULT SYSUTCDATETIME()	Audit trail.

4.2 *Players* Links a User to their Fortnite identity used for FortniteAPI.io lookups.

Column	Type	Constraint	Purpose
Player Id	INT (Identity)	PK	Unique player identifier.
User Id	INT	FK → Users.UserId, UNIQUE	Enforces the 1:1 link to a User account.
Fortnite Username	NVARCHAR(100)	UNIQUE, NOT NULL	Epic username; key used to call FortniteAPI.io.
Game	NVARCHAR(50)	DEFAULT 'Fortnite'	Reserved for possible multi-title support later.
Team	NVARCHAR(100)	NULL	Optional organization/team tag.
Created Date	DATETIME2	DEFAULT SYSUTCDATETIME()	Audit trail.

4.3 *Stats* — Holds the latest synced performance snapshot for a Player.

Column	Type	Constraint	Purpose
StatId	INT (Identity)	PK	Unique stats record identifier.
PlayerId	INT	FK → Players.PlayerId, UNIQUE	Enforces one live Stats row per Player.
Eliminations	INT	CHECK >= 0	Total eliminations from FortniteAPI.io.
Wins	INT	CHECK >= 0	Total wins.
Accuracy	DECIMAL(5,2)	CHECK >= 0	Shot accuracy percentage.
KDRatio	DECIMAL(6,2)	CHECK >= 0	Kill/Death ratio.

MatchesPlayed	INT	CHECK >= 0	Total matches played.
WinRate	DECIMAL(6,2)	COMPUTED, PERSISTED	Auto-derived from Wins / MatchesPlayed — never entered manually, so it can't drift out of sync.
LastUpdated	DATETIME2	DEFAULT SYSUTCDATETIME()	Timestamp of the most recent sync.

4.4 Recommendations — Stores the rule-based coaching output generated after each stats sync.

Column	Type	Constraint	Purpose
Recommendation Id	INT (Identity)	PK	Unique recommendation identifier.
Player Id	INT	FK → Players.PlayerId	Links each tip back to its player; not unique — a player can have many.
Recommendation Text	NVARCHAR(1000)	NOT NULL	The generated coaching message.
Created Date	DATETIME2	DEFAULT SYSUTCDATETIME()	Preserves coaching history in order.

5. Keys & Referential Integrity

- Every table uses a surrogate INT IDENTITY primary key rather than a natural key, so FortniteUsername or Email can change without breaking relationships.
- UNIQUE constraints on Players.UserId and Stats.PlayerId are what actually enforce the 1:1 relationships — a foreign key alone only enforces 1:many.
- All foreign keys cascade on delete (ON DELETE CASCADE): removing a User cleans up their Player, Stats, and Recommendations automatically, avoiding orphaned rows.
- A UNIQUE constraint on Players.FortniteUsername prevents two accounts from linking the same Epic username.

6. Normalization

Form How the schema satisfies it

- 1NF Every column holds a single atomic value (e.g., no comma-separated stat lists); each row is uniquely identified by its primary key.
- 2NF Every table has a single-column surrogate key, so no partial dependency on part of a composite key is possible.

Form How the schema satisfies it

3NF Non-key columns depend only on the primary key — e.g., WinRate is derived (computed column) rather than stored redundantly and manually kept in sync with Wins/MatchesPlayed.

7. Indexing Strategy

- Clustered index on each table's identity primary key (default).
- Unique index on Users.Email — used on every login lookup.
- Unique index on Players.FortniteUsername — used on every stats sync call to FortniteAPI.io.
- Composite non-clustered index on Recommendations (PlayerId, CreatedDate DESC) — supports the dashboard's "most recent coaching tips" query without a table scan.

8. Why This Design ?

Decision	Reasoning
Overwrite Stats instead of versioning it	The dashboard only needs the current snapshot; keeping history in Stats would require a separate time-series design not justified by current requirements.
Keep Recommendations append-only	Coaching value comes from seeing how advice changed as stats improved, so unlike Stats this table must retain every row.
Computed Win Rate column	Matches the acceptance criteria ("win rate is calculated automatically after sync") and removes a class of bugs where the app forgets to recompute it.
Surrogate keys everywhere	Epic usernames and emails can change; surrogate keys keep foreign key relationships stable regardless.
Four tables only	Mirrors the constraint from Requirement Analysis: a small semester-length project should avoid speculative tables (e.g., no separate GameMode table yet, since modes are handled at the application/enum level for now).

9. Data Flow Through the Schema Register/Login (Users) → Link Fortnite Account (Players) → Sync Stats via FortniteAPI.io (Stats upserted) → Recalculate Trends (WinRate computed column) → Generate Recommendation (new row in Recommendations) → Dashboard reads Stats + latest Recommendations.

10. Mapping to the Application Layer (EF Core)

- Each table maps 1:1 to a POCO entity class (User, Player, Stats, Recommendation) under Code-First conventions.
- Navigation properties mirror the relationships: Player.User, Player.Stats, Player.Recommendations.

- The Fluent API in `ApplicationDbContext.OnModelCreating` configures the UNIQUE constraints and computed column, since data annotations alone can't express a persisted computed column.
- Migrations (`dotnet ef migrations add`) generate and version this schema, keeping SQL Server and the C# model in sync.

11. Constraints & Assumptions Carried Forward

- Single SQL Server database instance — no sharding or read replicas needed at this scale.
- Stats sync frequency is bounded by `FortniteAPI.io`'s free-tier rate limit (10 requests/min), so no high-write-throughput design is required.
- Coaching remains rule-based; the schema does not assume any ML model fields, keeping Recommendations simple (text + timestamp).
- Schema is intentionally forward-compatible: swapping the rule-based engine for an AI/LLM API later would not require changing Recommendations' structure.